Document Retrieval Copilot

Agentic RAG Pipeline on AWS Bedrock

Policy & Claims Document Intelligence Workers' Compensation Insurance

Document Status	**APPROVED**
Version	1.0.0
Author	Senior Software Engineering Team
Date	April 2026
Division	Insurance Technology Workers' Compensation
Infrastructure	Amazon Web Services (AWS) Air-Gapped Deployment
Users Served	200,000+ Policyholders + Internal Claim Handlers

AWS Bedrock · FAISS · Agentic RAG · LangChain · React UI

1. Executive Summary

This document presents the software design for the Document Retrieval Copilot an AI-powered, agentic Retrieval-Augmented Generation (RAG) pipeline deployed entirely on Amazon Web Services (AWS) for a Workers' Compensation insurance platform serving more than 200,000 policyholders.

Prior to this system, claim handlers and policyholders were required to navigate multiple disconnected services and applications including separate policy management portals, claims systems, and document repositories to locate relevant policy and claim documents. This fragmented experience produced significant retrieval latency, handler inefficiency, and policyholder frustration.

The Copilot consolidates document retrieval into a single conversational interface. The core pipeline ingests policy and claims documents, classifies them using an agentic RAG architecture backed by FAISS vector embeddings on AWS Bedrock, and surfaces results through a chatbot UI accessible to both internal claim handlers and external policyholders.

Outcome	Result
Document retrieval improvement	43% reduction in time-to-document for claim handlers and policyholders
Users served	200,000+ policyholders + internal claim handler workforce
Deployment model	Fully air-gapped AWS environment physically separated from main company databases
Document pipeline	Ingestion → Classification → Embedding → Agentic RAG Retrieval → Chat UI
Security posture	Zero-trust, least-privilege, encrypted, no direct connectivity to production systems

2. Problem Statement

2.1 Background

Workers' Compensation insurance operations generate enormous volumes of structured and unstructured documents: policy declarations, endorsements, audit schedules, loss runs, adjuster notes, medical records, claim determinations, and settlement documents. These documents are spread across multiple siloed systems with no unified retrieval surface.

2.2 Pain Points

- Claim handlers navigate an average of 4–6 separate applications to assemble context for a single claim inquiry, consuming 15–30 minutes per session.
- Policyholders contacting service centers for document retrieval are placed on hold while agents locate records across disconnected systems.
- No semantic search capability exists all retrieval is keyword- or ID-based, producing low recall on natural language queries.
- Document classification is manual; misclassified documents are frequently surfaced in wrong contexts, undermining handler confidence.
- No conversational interface exists; every retrieval is a discrete transactional lookup with no context continuity.
- Compliance officers lack a unified audit trail for document access events across systems.

2.3 Impact Quantification

Pain Point	Frequency	Business Cost
Multi-app navigation per claim inquiry	Every inquiry	15–30 min handler time per session
Policyholder hold time for doc retrieval	High	Elevated handle time, CSAT score degradation
Manual document classification errors	~12% error rate	Wrong-context retrieval, re-work cycles
No semantic search	All queries	Low recall, missed relevant documents
No audit trail across systems	All access events	Compliance exposure, audit findings

2.4 Security Requirement

The solution must be deployed in an environment completely isolated from the main company database infrastructure. No direct network path, credential sharing, or data pipeline shall exist between the Copilot environment and production systems. Documents are replicated to the Copilot environment via a secure, one-way transfer mechanism.

3. Goals and Non-Goals

3.1 Goals

- Deploy a conversational document retrieval interface serving 200,000+ policyholders and internal claim handlers via a unified chatbot UI.
- Achieve a measurable 40%+ reduction in document retrieval time versus the pre-existing multi-application navigation baseline.
- Implement agentic RAG with FAISS vector store and AWS Bedrock embeddings for high-recall semantic document retrieval.
- Automate document classification at ingestion using an LLM-based classification agent.
- Deploy the entire system in an air-gapped AWS environment with zero direct connectivity to production company databases.
- Provide role-differentiated access: policyholders see only their own documents; claim handlers see documents scoped to their assigned claims.
- Maintain a complete, immutable document access audit trail for compliance review.
- Achieve 99.9% system availability with horizontal scalability to handle peak billing and renewal periods.

3.2 Non-Goals

- The Copilot does not write back to any production Policy or Claims System it is strictly read-access, retrieval-only.
- Document creation, editing, or e-signature workflows are out of scope for this release.
- Real-time bidirectional sync with production systems is not a design requirement; near-real-time one-way ingestion is sufficient.
- Claims adjudication decisions and premium calculations are outside the scope of this system.
- Mobile native applications (iOS/Android) are deferred to a future release.

4. System Architecture

4.1 High-Level Architecture Air-Gapped AWS Deployment

The system is partitioned into two completely separated AWS environments connected only by a secure, one-way data transfer mechanism. The Production Environment hosts source-of-truth policy and claims data. The Copilot Environment is a fully independent AWS account with no inbound network path from the Copilot to Production systems.


DOCUMENT RETRIEVAL COPILOT AWS ARCHITECTURE
Air-Gapped Deployment | Workers' Comp Insurance
PRODUCTION ENVIRONMENT (AWS Acct A)
[Company Production VPC]
Policy Management System
Claims System
Document Repository
ONE-WAY TRANSFER
(S3 Cross-Acct Repl)
No return path
COPILOT ENVIRONMENT (AWS Acct B)
[Air-Gapped No inbound from Prod]
INGESTION PIPELINE
S3 Document Landing Zone
→ AWS Lambda (Trigger)
→ Textract (OCR/Extraction)
→ Classification Agent (Bedrock LLM)
→ Chunking & Embedding (Bedrock)
→ FAISS Vector Store (EC2/EFS)
→ Metadata Store (DynamoDB)
AGENTIC RAG PIPELINE
API Gateway (WAF + Auth)
→ Orchestrator Lambda (LangChain)

4.2 AWS Infrastructure Components

AWS Service	Layer	Purpose
Amazon S3 (Landing Zone)	Ingestion	One-way cross-account document replication destination; source of truth for raw documents in Copilot env.
AWS Lambda	Compute	Ingestion trigger, classification orchestration, RAG pipeline orchestration, citation validation
Amazon Textract	Ingestion	OCR and structured data extraction from PDF, TIFF, and scanned claim documents
AWS Bedrock	AI/ML	Titan Embeddings v2 for vector generation; Claude 3 Sonnet for classification, reranking, response synthesis
FAISS (EC2 + EFS)	Vector Store	In-memory FAISS index hosted on EC2 with EFS persistence; supports cosine similarity ANN search at scale
Amazon DynamoDB	Metadata	Document metadata store: doc_id, type, policyholder_id, claim_id, classification label, S3 URI, embedding_id
Amazon ElastiCache (Redis)	Session	Conversation context and session state for the chatbot; 24-hour TTL per session
Amazon API Gateway	API	REST API for query requests; WebSocket API for real-time streaming responses to chatbot UI
Amazon Cognito	Auth	User pools for policyholder auth; identity pools for claim handlers with SAML federation to corp IdP
Amazon CloudFront	CDN/UI	Global CDN for React SPA; WAF integration for DDoS protection and bot mitigation
AWS KMS	Security	Customer-managed keys for S3, DynamoDB, ElastiCache, EFS encryption at rest
AWS Secrets Manager	Security	Bedrock API configuration, internal service credentials; auto-rotation enabled
AWS CloudTrail	Audit	Immutable audit log of all API calls, document access events, and admin actions
Amazon CloudWatch + X-Ray	Observability	Distributed tracing across pipeline; custom metrics per agent stage; dashboards and alarms
AWS WAF + Shield	Network Sec	OWASP ruleset on API Gateway and CloudFront; Shield Standard for DDoS at edge
Amazon EventBridge	Orchestration	Triggers ingestion pipeline on S3 object creation; schedules FAISS index refresh jobs

5. Document Ingestion Pipeline

5.1 Ingestion Flow Overview

Documents flow from Production systems into the Copilot environment via a strictly one-way S3 cross-account replication mechanism. Once in the Copilot landing zone, an event-driven pipeline processes each document through extraction, classification, chunking, embedding, and index registration.

5.2 Document Classification Agent

The classification agent is a LangChain tool-calling agent powered by Amazon Bedrock (Claude 3 Sonnet). It receives extracted text and document metadata, applies a multi-label classification schema, and returns a structured classification result with a confidence score and rationale.

```
Classification Agent Prompt Template:


System: You are a document classification expert for Workers' Compensation insurance.
        Classify the provided document into exactly one of the following categories:
        POLICY_DECLARATION, ENDORSEMENT, CLAIM_REPORT, MEDICAL_RECORD,
        ADJUSTER_NOTE, LOSS_RUN, SETTLEMENT, AUDIT_SCHEDULE, CORRESPONDENCE.
        Return a JSON object: { "label": "...", "confidence": 0.0-1.0, "rationale": "..." }


Tools available:
  - lookup_policy_schema(policy_id)     # Verify doc matches known policy
  - lookup_claim_schema(claim_id)       # Verify doc matches known claim
  - check_regulatory_keywords(text)    # Flag state-specific regulatory terms
```

5.3 Chunking Strategy

Parameter	Value	Rationale
Chunk size	512 tokens	Balances semantic coherence with embedding precision for insurance documents
Overlap	128 tokens	Prevents context loss at chunk boundaries for multi-page policy sections
Splitting strategy	Semantic (sentence-aware)	Avoids splitting mid-sentence; preserves clause integrity for legal documents
Metadata per chunk	doc_id, page, chunk_idx, doc_type	Enables citation with page-level precision in chatbot responses
Embedding model	Bedrock Titan Embeddings v2	1536-dimension vectors; optimized for long-form document retrieval tasks

6. FAISS Vector Store Design

6.1 FAISS Architecture

The FAISS (Facebook AI Similarity Search) vector index is hosted on a dedicated EC2 instance (r6i.4xlarge for memory-optimized ANN search) with the index persisted to Amazon EFS for durability and multi-instance sharing. A lightweight FastAPI service wraps the FAISS operations and exposes an internal REST API, accessible only within the Copilot VPC private subnet.

```
FAISS Service Architecture:


  EC2 (r6i.4xlarge, 128 GB RAM)  Primary FAISS Node
  ├── FAISS IndexIVFFlat (cosine similarity, nlist=1024)
  ├── In-memory index: ~200M vectors at peak (200K policyholders × avg 1000 chunks/
policyholder)
  ├── EFS mount: /mnt/faiss-index/ persistent index snapshots every 15 min
  ├── FastAPI service (port 8080, internal only, Security Group: copilot-faiss-sg)
  │     ├── POST /index   add vectors (called by ingestion pipeline)
  │     ├── POST /search  ANN search (called by retrieval agent)
  │     └── POST /delete  soft-delete by doc_id (called by admin pipeline)
  └── Auto Scaling Group: min 2, max 6 nodes; scale on CPU > 70%


  Index Type:    IndexIVFFlat with cosine similarity metric
  Dimensions:    1536 (Titan Embeddings v2)
  nprobe:        64 (search breadth parameter tuned for recall vs latency)
  Top-K default: 20 candidates (reranked to top 5 for response synthesis)
  P95 latency:   < 120 ms for 10M+ vector index
```

6.2 FAISS Index Partitioning

To enforce strict data isolation between policyholders, the FAISS index is logically partitioned by policyholder_id namespace. The retrieval agent pre-filters the search space by policyholder_id metadata before executing ANN search. This ensures a policyholder query never returns documents belonging to another policyholder.

Property	Design Decision
Isolation model	Metadata pre-filter on policyholder_id before FAISS search; enforced at service layer, not UI layer
Claim handler scope	Retrieval filtered to claim_ids assigned to authenticated handler; no cross-claim access
Admin scope	Separate privileged IAM role; all admin queries logged to dedicated CloudTrail stream
Index refresh	Incremental: new vectors appended on ingestion; full rebuild scheduled weekly (EventBridge cron)
Deletion handling	Soft-delete flag in DynamoDB; hard removal on weekly rebuild to comply with data retention policies

7. Agentic RAG Pipeline

7.1 Pipeline Overview

The retrieval pipeline is a multi-agent LangChain system orchestrated by an AWS Lambda function. Each incoming user query passes through four sequential agents before a response is returned to the chatbot UI via WebSocket streaming.

Document Retrieval Copilot AWS Bedrock Agentic RAG

7.2 Agent Specifications

Agent	Model	Avg Latency	Fallback
Query Understanding	Claude 3 Haiku (Bedrock)	< 400 ms	Passthrough + keyword fallback
Retrieval	FAISS + Titan Embeddings v2	< 150 ms	Keyword metadata search (DynamoDB)
Reranker	Claude 3 Haiku (Bedrock)	< 600 ms	Skip reranking, use FAISS scores
Response Synthesizer	Claude 3 Sonnet (Bedrock)	1.5–3 s (streaming)	Return ranked chunk excerpts directly

7.3 Conversation Context Management

The chatbot maintains multi-turn conversation context using ElastiCache (Redis). Each session stores the last 10 conversation turns (user query + agent response) as a compressed context window injected into subsequent prompts. This enables coherent follow-up questions without re-querying already established document context.

```
Session State Schema (Redis key: session:{cognito_sub}):


{
  "session_id":      str,           // UUID v4
  "user_id":         str,           // Cognito sub
  "user_role":       str,           // POLICYHOLDER | CLAIM_HANDLER | ADMIN
  "auth_scope": {
    "policyholder_id": str,
    "claim_ids":       list[str]    // For CLAIM_HANDLER role
  },
  "turns": [                        // Last 10 turns (FIFO ring buffer)
    { "role": "user",      "content": str, "timestamp": datetime },
    { "role": "assistant", "content": str, "citations": list[Citation] }
  ],
  "ttl": 86400                      // 24-hour expiry
}
```

8. Chatbot UI Design

8.1 UI Architecture

The chatbot frontend is a React Single Page Application (SPA) hosted on Amazon S3 and distributed globally via Amazon CloudFront with AWS WAF integration. The UI communicates with the backend via WebSocket API Gateway for real-time streaming responses, and REST API Gateway for authentication flows, session management, and document download pre-signed URL generation.

8.2 User Roles & Experience

Role	Auth Method	UI Capabilities
Policyholder	Cognito User Pool (email + MFA)	Query own policy docs, claim status docs, correspondence. View citations with page links. Download documents via pre-signed S3 URLs (15-min expiry).
Claim Handler	Cognito + SAML (Corp IdP)	Query all documents within assigned claim portfolio. Cross-policy search within scope. Export retrieval session transcript. Access adjuster notes and medical records.
Admin	Cognito + SAML + MFA Hardware Token	Full document access. Manage ingestion pipeline. View audit logs. Trigger index rebuild. No production system access (air-gap enforced).

8.3 UI Component Architecture

```
CloudFront Distribution (copilot.internal.company.com)
       │  Origin: S3 bucket (react-spa-copilot)
       │  WAF: OWASP rules + rate limiting (500 req/min per IP)
       ▼
React SPA Components:
├── AuthProvider (Cognito Amplify SDK)
│     ├── LoginPage (username + MFA)
│     └── SAMLRedirect (claim handler corp SSO)
├── ChatInterface
│     ├── MessageList (streamed response rendering)
│     ├── CitationPanel (source document cards with page refs)
│     ├── DocumentViewer (inline PDF preview, S3 pre-signed URL)
│     └── QueryInput (voice-to-text, autocomplete for policy/claim IDs)
├── SessionSidebar
│     ├── ConversationHistory (last 10 sessions)
│     └── SavedDocuments (bookmarked retrievals)
└── AdminPanel (role-gated)
      ├── IngestionMonitor (pipeline status)
      └── AuditLogViewer (CloudTrail events)
```

8.4 WebSocket Communication Protocol

```
WebSocket API Gateway Route: $connect / $disconnect / query


Client → Server (query message):
{
  "action":     "query",
  "session_id": "uuid",
  "query":      "What is my current policy deductible?",
  "auth_token": "Cognito JWT"
}


Server → Client (streamed response):
{ "type": "token",    "content": "Your current..." }   // Streamed tokens
{ "type": "citation", "doc_id": "...", "page": 4 }     // Inline citations
{ "type": "done",     "session_id": "uuid" }           // Stream complete
{ "type": "error",    "code": "AUTH_SCOPE_VIOLATION" } // Error event
```

9. Security Architecture

9.1 Air-Gap Design Core Security Principle

The Copilot environment operates in a completely separate AWS account (Account B) from the production company infrastructure (Account A). There is no VPC peering, no Transit Gateway connection, no Direct Connect cross-account path, and no IAM role trust relationship that would allow any principal in Account B to access Account A resources. The only data flow is a strictly one-way S3 Cross-Account Replication from Account A to Account B.

```
Air-Gap Enforcement Rules:


1. No VPC peering or Transit Gateway between Acct A and Acct B

2. S3 replication role in Acct A:

     - Permissions: s3:ReplicateObject, s3:ReplicateDelete (target bucket only)

     - NO s3:GetObject from Acct B write-only, no read-back path

3. Acct B IAM SCPs (Service Control Policies):

     - Deny: ec2:CreateVpcPeeringConnection

     - Deny: ram:CreateResourceShare (prevents cross-account sharing)

     - Deny: any action targeting Acct A resource ARNs

4. Acct B has no outbound internet access except:

     - AWS Bedrock (via PrivateLink)

     - Amazon Textract (via PrivateLink)

     - NAT Gateway → allow list only (package registries, CRL endpoints)

5. Network egress monitoring: GuardDuty + VPC Flow Logs to CloudWatch
```

9.2 Security Controls Matrix

Layer	Control	Implementation	Detail
Network	VPC Isolation	Private subnets only	All compute in private subnets; no public IPs assigned to any service
Network	PrivateLink	Bedrock, Textract, S3	All AWS service calls via VPC endpoints no internet traversal
Network	AWS WAF	CloudFront + API GW	OWASP Top 10; rate limit 500 req/min; geo-block non-US IPs

Network	Security Groups	Per-service micro-seg.	FAISS SG: port 8080, source = orchestrator Lambda SG only
Identity	Cognito MFA	All user pools	TOTP MFA required for policyholders; hardware token for admins
Identity	IAM Least Privilege	Per-Lambda role	Each Lambda has scoped role; no wildcard actions; STS temp creds
Identity	Cognito Scopes	JWT claims	policyholder_id and claim_ids bound in JWT; validated server-side per request
Data	KMS CMK	All data stores	S3, DynamoDB, ElastiCache, EFS all encrypted with customer-managed KMS keys
Data	TLS 1.3	All endpoints	Enforced on API Gateway, WebSocket, CloudFront, internal FAISS API
Data	PII Masking	Logs + Prompts	SSN, DOB, account numbers masked in CloudWatch logs and Bedrock prompt logs
Data	Pre-signed URLs	Document download	S3 pre-signed URLs for doc download; 15-minute expiry; no direct S3 access
Application	Auth Scope Enforce	FAISS pre-filter	policyholder_id injected into every FAISS query at service layer; cannot be overridden by user input
Application	Prompt Injection Guard	Input sanitization	User query sanitized before LLM prompt construction; system prompt protected from user-supplied content
Audit	CloudTrail	All API calls	S3 data events, DynamoDB access, Lambda invocations immutable log with integrity validation
Audit	Doc Access Log	Custom EventBridge	Every document retrieval event: user_id, doc_id, timestamp, query_text (hashed) → S3 audit log
Audit	GuardDuty	Threat detection	Anomalous API patterns, credential compromise, port scanning detection across Acct B
Audit	AWS Macie	S3 PII scan	Continuous scanning of ingested documents for unexpected PII patterns; alert on anomalies

9.3 IAM Role Definitions

```
ingestion-orchestrator-lambda-role:
  ├── s3:GetObject                (copilot-landing-zone/* read ingested docs)
  ├── textract:StartDocumentAnalysis, GetDocumentAnalysis
  ├── bedrock:InvokeModel         (Titan Embeddings + Claude 3 Sonnet classify)
  ├── dynamodb:PutItem            (document-metadata table)
  └── kms:GenerateDataKey, Decrypt (CMK for S3 + DynamoDB)


faiss-writer-role (EC2 instance profile):
  ├── efs:ClientMount             (faiss-index EFS)
  ├── cloudwatch:PutMetricData    (FAISS health metrics)
  └── kms:Decrypt                 (EFS CMK)


rag-orchestrator-lambda-role:
  ├── bedrock:InvokeModel         (Titan Embeddings + Claude 3 Haiku + Sonnet)
  ├── dynamodb:GetItem, Query     (document-metadata table read only)
  ├── elasticache:Connect         (Redis session store)
  ├── s3:GetObject                (invoice-artifacts for pre-signed URL gen)
  ├── s3:GeneratePresignedUrl     (document download)
  └── kms:Decrypt                 (CMK for all read ops)


chatbot-ui-role (Cognito Identity Pool):
  ├── execute-api:Invoke          (API Gateway scoped to /query endpoint)
  └── NO direct S3/DynamoDB/FAISS access all mediated by Lambda
```

10. End-to-End Data Flow

10.1 Document Ingestion Flow

```
Step 1:  New document created in Production Policy/Claims System (Acct A)
Step 2:  S3 Cross-Account Replication triggers object copied to
         s3://copilot-landing-zone-acct-b/{doc_type}/{policyholder_id}/{doc_id}.pdf
Step 3:  EventBridge rule fires on ObjectCreated → invokes IngestionOrchestrator Lambda
Step 4:  Lambda submits doc to Amazon Textract (async) extracts text + form fields
Step 5:  Textract completion event → Lambda resumes
Step 6:  Classification Agent (Bedrock Claude 3 Sonnet) classifies document
         → Returns: { label: "CLAIM_REPORT", confidence: 0.94 }
Step 7:  Text chunker splits extracted text into 512-token chunks with 128-token overlap
Step 8:  Bedrock Titan Embeddings v2 generates 1536-dim vector per chunk (batched)
Step 9:  FAISS Writer API called vectors indexed with metadata namespace {policyholder_id}
Step 10: DynamoDB metadata record written:
         { doc_id, policyholder_id, claim_id, doc_type, s3_uri, chunk_ids[], ingested_at }
Step 11: CloudWatch metric emitted: ingestion_success | ingestion_failure
Step 12: EventBridge event: document.ingested → downstream analytics pipeline
```

10.2 Query Retrieval Flow

Document Retrieval Copilot AWS Bedrock Agentic RAG

```
Step 1:  User types query in Chatbot UI → WebSocket message sent to API Gateway

Step 2:  API Gateway invokes RAG Orchestrator Lambda

Step 3:  Lambda validates Cognito JWT extracts user_id, role, policyholder_id, claim_ids

Step 4:  Session context loaded from ElastiCache (Redis) last 10 turns injected

Step 5:  Query Understanding Agent (Claude 3 Haiku):

         - Intent: DOCUMENT_LOOKUP

         - Entities: { claim_id: "CLM-2024-00441", doc_type: "MEDICAL_RECORD" }

         - Rewritten query: "medical records for workers comp claim CLM-2024-00441"

Step 6:  Rewritten query → Bedrock Titan Embeddings v2 → 1536-dim query vector

Step 7:  FAISS search: top-20 candidates, pre-filtered: policyholder_id=PH-98234

         AND claim_id IN [CLM-2024-00441] (auth scope enforced at FAISS service)

Step 8:  DynamoDB: enrich 20 candidates with doc metadata (type, page, s3_uri)

Step 9:  Reranker Agent (Claude 3 Haiku): cross-encode query vs 20 candidates

         → Top 5 chunks selected; 3 below threshold (score < 0.65) discarded

Step 10: Response Synthesizer (Claude 3 Sonnet): generate grounded response

         with top-5 chunks as context; citations mapped to source chunks

Step 11: Citation Validator: every factual claim verified against source chunks

         Unverified claims redacted before response transmission

Step 12: Response streamed token-by-token via WebSocket to Chatbot UI

Step 13: Conversation turn appended to Redis session (FIFO ring, max 10 turns)

Step 14: Audit log written: { user_id, policyholder_id, doc_ids_accessed, timestamp }
```

11. Scalability & Performance

11.1 Scale Targets

Metric	Target	Design Headroom
Policyholders	200,000+	500,000 (2.5×)
Concurrent chatbot sessions	5,000	15,000
Query response time (p95)	< 5 seconds end-to-end	< 3 seconds (optimized)
Documents indexed	50M+ vectors	200M+ (FAISS cluster mode)
Ingestion throughput	1,000 docs/hour	5,000 docs/hour
System availability	99.9%	99.95% (Multi-AZ)

11.2 Scaling Strategies

- Lambda Concurrency: RAG Orchestrator Lambda provisioned concurrency = 100; reserved concurrency = 1,000; scales to handle burst query load.
- FAISS Auto Scaling: EC2 Auto Scaling Group for FAISS nodes target tracking on CPU utilization > 70%; min 2 nodes (Multi-AZ), max 6 nodes.
- API Gateway WebSocket: Managed by AWS; no scaling configuration required; handles 100K+ concurrent connections natively.
- ElastiCache Cluster: Redis 7.x in cluster mode with 3 shards × 2 replicas; session read operations distributed across read replicas.
- DynamoDB On-Demand: Auto-scaling billing mode; no capacity planning required; handles burst ingestion events during policy renewal periods.
- CloudFront: Global edge caching for static SPA assets; 99th-percentile TTFB < 100 ms worldwide.

12. Observability & Monitoring

12.1 Key Metrics & Alerts

Metric	Source	Alert Condition
Query response time p95	X-Ray	Alert if > 8 seconds
FAISS search latency p99	FAISS FastAPI metrics	Alert if > 500 ms
Retrieval recall@5	Custom CloudWatch	Alert if < 0.80 over 1-hour window
Ingestion pipeline failures	Lambda metrics	Alert on any failure (PagerDuty P2)
Citation validation failures	Custom CloudWatch	Alert if > 2% of responses
Auth scope violations	Custom CloudWatch	Alert on any occurrence (P1 security incident)
Bedrock token spend	CloudWatch Billing	Alert if daily spend > budget threshold
Concurrent WebSocket sessions	API Gateway	Alert if > 4,000 (scale prep)

12.2 Dashboards

- Pipeline Health Dashboard: Ingestion throughput, classification accuracy (sampled), FAISS index size, embedding latency.
- Query Quality Dashboard: Retrieval recall, reranker score distribution, citation validation pass rate, hallucination event rate.
- User Experience Dashboard: Session count, p50/p95 query response time, chatbot UI error rate, session abandonment rate.
- Security Dashboard: Auth scope violation count, GuardDuty finding count, WAF blocked request rate, CloudTrail anomaly alerts.

13. Outcomes & Measured Results

13.1 Key Metrics Pre vs Post Deployment

Metric	Before	After
Time-to-document (claim handler, p50)	18 minutes	10.3 minutes (43% reduction)
Time-to-document (policyholder, p50)	22 minutes (incl. hold)	< 2 minutes self-serve
Apps navigated per inquiry	4–6 applications	1 (Copilot chatbot)
Document classification accuracy	~88% (manual)	96.2% (automated LLM)
Retrieval recall@5 (semantic search)	N/A (keyword only)	0.87 measured recall@5
Policyholder self-serve rate	< 10%	67% (as of Q1 2026)
Compliance audit findings (doc access)	2 per year	0 since deployment

13.2 Qualitative Outcomes

- Claim handlers report significantly reduced cognitive load a single conversational interface replaces navigation across 4–6 disconnected applications.
- Policyholders with self-serve access report higher satisfaction scores in post-interaction surveys, citing immediacy and accuracy of document retrieval.
- Compliance team confirmed that the unified audit trail (CloudTrail + custom document access log) resolved the primary finding from the prior year's market conduct examination.
- The air-gapped architecture was specifically cited by InfoSec leadership as a model for future AI assistant deployments, demonstrating that LLM-powered tools can be deployed without exposing production data systems.

14. Future Work & Roadmap

Phase	Feature	Description
v1.1	Bedrock Knowledge Bases	Migrate from self-managed FAISS to Amazon Bedrock Knowledge Bases (managed vector store) to reduce operational overhead.
v1.2	Multi-modal documents	Extend ingestion to support TIFF medical imaging and handwritten adjuster notes via Textract + Claude 3 vision.
v1.3	Proactive alerts	EventBridge-driven proactive notifications: "Your claim document has been updated" pushed to policyholder chatbot.
v2.0	Mobile native app	React Native iOS/Android app with biometric auth (FaceID/ fingerprint) replacing web chatbot for policyholders.
v2.1	Cross-LoB expansion	Extend document pipeline to General Liability and Commercial Auto lines with configurable classification schemas.
v2.2	Feedback-loop RAG	Collect implicit user feedback (query reformulation, session abandonment) to fine-tune retrieval ranking model.

15. Appendix

A. Glossary

Term	Definition
RAG	Retrieval-Augmented Generation LLM pattern that grounds responses in retrieved documents rather than parametric knowledge alone.
FAISS	Facebook AI Similarity Search open-source library for efficient approximate nearest-neighbor search over dense vector embeddings.
Agentic RAG	RAG architecture where discrete LLM agents handle sub-tasks (query understanding, retrieval, reranking, synthesis) as a coordinated pipeline.
Air-Gap	Network isolation where two environments have no direct connectivity path; data transfer occurs only via controlled, unidirectional mechanisms.
ANN	Approximate Nearest Neighbor algorithm for finding vectors most similar to a query vector in large high-dimensional spaces, trading exactness for speed.
Bedrock	AWS managed AI service providing access to foundation models (Claude, Titan, etc.) via API within the AWS security boundary.
Cognito	AWS managed user identity and access management service; supports user pools, identity pools, OAuth2, and SAML federation.
Cross-Encoder	Reranking model that scores query-document pairs jointly (vs bi-encoder which embeds separately); more accurate but higher latency.
Citation Validator	Agent that verifies each factual claim in a synthesized response maps to a specific retrieved chunk; prevents hallucination in user-facing output.
SPA	Single Page Application web application that loads once and updates dynamically without full page reloads; built with React in this system.

B. References

- AWS Bedrock Documentation: https://docs.aws.amazon.com/bedrock/
- FAISS Documentation: https://faiss.ai/
- LangChain Documentation: https://python.langchain.com/docs/
- Amazon Textract Documentation: https://docs.aws.amazon.com/textract/
- AWS Well-Architected Framework Security Pillar
- NAIC Insurance Data Security Model Law (MDL-668)
- CIS AWS Foundations Benchmark v2.0
- OWASP Top 10 for LLM Applications (2025)

C. Revision History

Version	Date	Author	Changes

0.1	Nov 2025	Architecture Team	Initial draft architecture proposal
0.5	Jan 2026	Security Team	Air-gap design + security controls added
0.8	Feb 2026	ML Engineering	FAISS design, agentic RAG pipeline finalized
0.9	Mar 2026	UI Engineering	Chatbot UI architecture + WebSocket protocol
1.0	Apr 2026	Sr. SWE / Tech Lead	Final review, outcomes data, approved